

FastVOI: Efficient Utility Elicitation During Negotiations

Yasser Mohammad^{1,2}[0000–0003–2272–7254] and Shinji Nakadai^{1,3}[0000–0003–1652–838X]

¹ AIST, Tokyo, Japan

² Assiut University, Egypt

³ NEC DataScience Research Laboratories, Tokyo, Japan
y.mohammad@aist.go.jp, s-nakadai@aist.go.jp

Abstract. Autonomous Negotiation is a promising technology that allows individuals and institutions to reduce the burden and cost of negotiating win-win agreements. A common challenge in practical applications is the inability or high cost of finding the utility value for each possible outcome of the negotiation before it even starts. Earlier work on utility elicitation during negotiations tried to avoid the need of full revelation of the utility function to the agent by interleaving elicitation and negotiation actions. This paper proposes an efficient elicitation algorithm that allows the agent to achieve similar utility at orders of magnitude higher speed compared with the state-of-the-art algorithm.

Keywords: Autonomous Negotiation · Utility Elicitation.

1 Introduction

Automatic negotiation is attracting more attention from the research community in recent years especially given the rise of AI, machine learning systems and the Internet of Things (IoT) that promise to automate most repetitive aspects of our lives. Recent applications of automatic negotiation include permission management in IoT systems, Wi-Fi channel assignment [5], agriculture supply chain support, and providing feedback for student negotiation skills [8].

Most of this work assumes that the negotiation agent has perfect knowledge of the utility function of the person/entity it is representing during the negotiation. While this can be the case in some limited situations, in many real-world scenarios; it is not possible to have perfect apriori knowledge of this utility function.

Utility elicitation was studied extensively in the decision support community [6]. Most of this work focuses on the problem of eliciting the utility function of an actor for several possible outcomes of the decision process. Negotiation adds a new complexity to this problem because during negotiation it is not enough to know the utility value of some outcome for the user in order to propose/accept it but it is also essential to judge the probability that this outcome/offer is also acceptable by the partner(s) in the negotiation.

More recently, few works have reported systems for utility elicitation *during* negotiation [2, 3, 10]. These systems build upon earlier work in *preference elicitation* in the decision support domain [4] while taking into account the specific features of negotiation.

The *optimal elicitation algorithm* [2] assumes that the actor (user) can be queried to provide *exact* utility values for different outcomes. This can be achieved using several possible elicitation strategies that does not require actual assignment of a numeric value to any outcome by the user so it is not restrictive. Nevertheless, this form of *deep* elicitation for each outcome considered is time-consuming and would lead to high levels of elicitation bother to the user that can sometimes be avoided. A *shallow* version of this algorithm was recently proposed that also uses a heuristic to manage avoid offering outcomes that turn out to have low utilities in the beginning of the negotiation [10].

The *Optimal Query Agent (OQA)* [3] avoids the problem of deep elicitation by assuming that a predefined set of possible queries are available that can reduce the uncertainty in the probability distribution of utility values for a given outcome. The system selects the optimal query at each point as the one maximizing the value of information which is the difference between the *expected utility* if the answer to the query is known compared with it if the answer is not known. The main disadvantage of this approach is its high computational cost which increases linearly with both the outcome space size and the number of available queries.

This paper proposes an efficient value of information based algorithm (FastVOI) that is shown to achieve the same utilities as OQA and provide a performance on-par with an ideal agent with full knowledge of the utility function while reducing the complexity to $O(n \log n)$ where n is the number of outcomes.

2 Problem Setting

A negotiation session is conducted between multiple agents representing self-interested actors over a set of issues. Issues can have discrete or continuous values. Every possible assignment of a value to each issue is called an *outcome* and during negotiation it may also be called an *offer*. Ω denotes the – possibly uncountable – set of all outcomes and ω_i indicates a member of this set. If an agreement is reached, the agreed upon outcome is called a *contract* (ω_c). Each actor a is assumed to be self-interested with some internal *utility function* $\tilde{U}_a : \Omega \rightarrow [0, 1]$ that assigns a numeric value (assumed to be normalized to the range 0 to 1 in this work) to every possible outcome.

The actor wishes to maximize the utility she receives from the negotiation through the behavior of its representing agent. Every actor also has a predefined *reserved value* (R_a) that she receives if the negotiation was broken either due to timing out or explicitly by one of the agents.

Negotiation sessions are conducted in *rounds* in which different outcomes are offered/judged by the agents according to some negotiation *protocol*. Negotiation protocols can be moderated with a moderator-agent that have a special

role or unmoderated with all agents having similar roles. Several negotiation protocols have been proposed over the years. They can either be mediated [7] or un-mediated [1]. This work utilizes the Stacked Alternating Offers Protocol (SOAP) [1] but is not limited to it.

To provide an incentive for the agents to concede from the outcome with maximum utility, a time-limit is usually applied to the negotiation and the session is broken (times-out) automatically if no agreement is reached within a predefined real-time (T) or number of rounds limit (K). A constant cost δ_a is incurred by every agent at the end of each negotiation round to encourage early agreement. Moreover, any other costs incurred by the agent during the negotiation (e.g. bother cost to the actor) (C_a) is always subtracted from its final utility.

For the rest of this paper, k represents the round number, $t \equiv k/K$ is the relative time, and $n \equiv |\Omega|$ is the total number of outcomes. Superscripts represent round number. The proposed system can trivially be applied to cases with any combinations of real-time and round-limit constraints as well as to cases with constant or discounted negotiation cost.

The SOAP protocol works as follows: An ordering of the agents is defined. We assume — without loss of generality — that it is the same as the agent index a .

The first agent starts the negotiation by *offering* an outcome ω^0 which is visible to all other agents. The next agent either *accepts* the offer, ends the negotiation, or proposes a new offer.

This process is continued until one of the following stopping criteria is met:

- **Agreement:** An offer ω_c is accepted by all agents which is declared as the *contract*. Each agent a receives a utility of $\tilde{U}_a(\omega_c) - k \times \delta_a - C_a$.
- **Timeout:** A predefined number of offer exchanges/rounds (K) or a predefined number of seconds T has passed since the beginning of the negotiation. Each agent a receives a utility of $R_a - K\delta_a - C_a$.
- **Failure:** Some agent ends the negotiation when it has the chance to respond to some offer. Each agent a receives a utility of $R_a - k\delta_a - C_a$.

The agent a representing actor a is not assumed to have access to $\tilde{U}_a$ but have access to a joint probability distribution over possible outcome-utility values $\hat{U}_a^0 : [0, 1]^{|\Omega|} \rightarrow [0, 1]$. Hereafter, when the agent considered is clear from the context, the subscript a will be dropped.

In utility elicitation scenarios, the agent has the ability to *ask* the actor questions that would reduce this source of uncertainty. A reply from the actor can in general be modeled as a mapping from some joint probability distribution over outcome utilities to a new one with smaller spread of values (e.g. variance).

More formally, a negotiation scenario with elicitation adds a set of queries $Q^k \equiv \{q_l^k\}$ where each query q_l^k is defined as a set of tuples (r_{li}, p_{li}, c_{li}) and r_{li} is answer i for query l , p_{li} is the probability of getting this answer, and c_{li} is the cost of getting it (e.g. bother cost to the actor in utility units). In most cases the cost is the same for all answers and $c_{li} = c_{lj} = c_l \forall i, j$. Hereafter, m is the number of queries $|Q|$.

An answer r_{li} can generally be modeled as a function that receives a joint utility distribution $\hat{U}^k$ and returns another utility distribution $\hat{U}^{k+1}$ which has total lower spread/variance signaling a reduction of uncertainty. Elicitation during round k is the process of selecting one query q_l^k from Q^k , presenting it to the actor a , receiving a reply $r^k \in \{r_{li}\}$ then applying the transformation defined by that reply to the joint utility distribution: $\hat{U}^k \leftarrow r^k [\hat{U}^{k-1}]$

3 VOI Elicitation Algorithm: Optimal Query Agent

The Value-of-information (VOI) based algorithm for preference elicitation was first proposed by Chajewska [4] in the context of decision theory. A related algorithm explicitly designed for elicitation during negotiations was recently proposed by Baarslag and Kaisars [3]. Our proposed variant is based on this algorithm. This section provides a brief description of it.

All VOI variants assume that of an acceptance model $\mathcal{M} : \Omega \rightarrow [0, 1]$ is available to the agent. We use the same subscript/superscript rules defined in section 2 with the acceptance model.

A policy $\Psi \equiv \langle \omega_j | \omega_j \in \Omega \rangle$ of length D is an ordered sequence of outcomes (Ψ_i is the outcome at index i in Ψ).

Given an elicitation scenario and an acceptance model $\mathcal{M}$, the expected utility of following a policy Ψ will be:

$$EU(\Psi | \mathcal{M}, \tilde{U}) = \sum_{k=1}^{|\Psi|} \mathcal{M}(\Psi_k) \tilde{U}(\Psi_k) \prod_{j=1}^{k-1} (1 - \mathcal{M}(\Psi_j)). \quad (1)$$

Given that the agent has no access to the true utility of the actor $\tilde{U}$, it can only calculate the expectation of EU which is known as the expected expected utility [3]:

$$EEU(\Psi | \mathcal{M}, \hat{U}) = \sum_{k=1}^{|\Psi|} \mathbb{E}[\hat{U}_{\Psi_k}] \mathcal{M}(\Psi_k) \prod_{j=1}^{k-1} (1 - \mathcal{M}(\Psi_j)), \quad (2)$$

where, $\mathbb{E}[x]$ is the expected value of x .

An optimal policy π is a policy that maximizes EEU with respect to a given acceptance model and utility distribution:

$$\pi_{\mathcal{M}, \hat{U}} \equiv \arg \max_{\Psi} EEU(\Psi | \mathcal{M}, \hat{U}) \quad (3)$$

Baarslag and Kaisars [3] provided an efficient greedy algorithm for calculating $\pi_{\mathcal{M}, \hat{U}}$ that can calculate the optimal policy of length l given the one of length $l-1$ by extending it with ω_* where:

$$\omega_* \equiv \arg \max_{\omega} \mathbb{E}[\hat{U}_{\omega}] \mathcal{M}(\omega).$$

Given $\mathcal{M}, \hat{U}$, the elicitation process of the Optimal Query Agent (OQA) [3] proceeds by calculating the optimal policy π with the expected utility under this policy (eeu^*), then for each question $q \in Q$, an *optimal alternative policy* π_{qr} for each possible answer r is calculated with the associated eeu_{qr} as:

$$\pi_{qr} = \arg \max_{\psi} EEU \left(\Psi | \mathcal{M}, r(\hat{U}) \right). \quad (4)$$

The EEU of that question is then calculated as the weighted average of EEU of following the optimal policy after getting each answer with the associated cost subtracted. The VOI algorithm asks the question q^* that maximizes eeu_q as long as $eeu_{q^*} > eeu^*$ which means that asking this question would entail a positive value of information.

This approach to elicitation is promising as it puts few limitations on the kinds of questions being asked and provides a provably optimal solution under the aforementioned assumptions. Its main limitation is the high computational cost. Using the greedy algorithm for optimal policy calculation, the complexity of OQA is $O(nm)$. In general, the number of available queries will increase — at least — linearly with the number of outcomes because adding a new possible outcome will entail adding at least one relevant query about its utility if it is not already known. This leads to a quadratic complexity $O(n^2)$.

4 Proposed Algorithm

The main bottleneck in OQA calculations is finding the optimal policy associated with each answer. As this process is repeated for each answer of each query, it is beneficial to optimize it.

The process proposed by Barslaag et al. [3] detailed in section 3 can find the optimal policy from scratch in $O(n)$ operations. We use a small variant of this process to find the optimal policy for the current distribution $\hat{U}$ keeping some extra information to speedup later calculations. Nevertheless, we propose an update process that can find the optimal policy associated with each answer in $O(\log n)$ given this modified initialization step.

To make it possible to update the EEU value associated with each question in $O(\log n)$ operations, the optimal policy is not stored as a list but as a sorted list implemented using a TREAP data structure. A TREAP data structure combines a tree and a heap leading to logarithmic search and access times by ensuring that the tree is balanced due to the randomization of the priority used to order it.

For each outcome ω , we can calculate the corresponding sorting key as:

$$eu_{\omega} = \mathbb{E} \left[\hat{U}_{\omega} \right]. \quad (5)$$

An *inverse* TREAP that uses the outcome as key and the corresponding expected utility (eu) as value is also initialized to speedup localization of outcomes within the first TREAP.

To simplify the notation, we define the cumulative sum and product at element i in Equation 2 as:

$$P_i \equiv \prod_{j=1}^{i-1} 1 - \mathcal{M}(\pi_j), \quad (6)$$

$$S_i \equiv \sum_{k=1}^i \mathcal{M}(\omega_k) \mathbb{E}[\hat{U}_k] \prod_{j=1}^{k-1} 1 - \mathcal{M}(\pi_j) = \sum_{k=1}^i M_k eu_k P_k, \quad (7)$$

where $M_k \equiv \mathcal{M}(\pi_k)$, and $S_0 = 0$ and P is defined up to and including P_{n+1} .

Given these definitions, it is clear that $EEU = S_m$ (see Equations 2, 7). It can be shown that the policy calculated in this manner is actually optimal.

Given that we have the initialized optimal policy, if the expected utility of outcome π_k changed from eu_k to eu'_k without a change to the acceptance model, it can be shown that the new expected expected utility can be found as:

$$EEU' = \begin{cases} EEU - M_k eu_k + M_k eu'_k & j = k \\ EEU - S_k + S_{j-1} + M_k eu'_k P_j + (S_{k-1} - S_{j-1})(1 - M_k) & j < k \\ EEU - S_j + S_{k-1} + \frac{S_j - S_k}{1 - M_k} + \frac{M_k u'_k P_{j+1}}{1 - M_k} & j > k \end{cases} \quad (8)$$

where j is the new location of π_k in the optimal policy after the change. The case when the opponent model changes is slightly more involved and will not be presented here to simplify the presentation as it does not occur during utility elicitation because replies to questions from the user can only alter the expected utility of some outcomes but we assume that the user has no knowledge about the opponent to change the acceptance model beyond whatever it was initialized to be.

Once we have the optimal policy π and associated EEU value, we can find the optimal policy associated with any answer π_{qr} by deleting the node containing that outcome from the optimal policy then re-inserting it with the new eu value as calculated from Equation 5 after replacing $\hat{U}_\omega$ with $r(\hat{U}_\omega)$. Both of these operations take logarithmic time. Moreover, the EEU value can be updated using Equation 8 in constant time. This means that updating the optimal policies associated with each response for all queries requires $O(m \log n)$.

The elicitation process requires one application of optimal policy initialization routine which takes $O(n \log n)$ operations. After that, whenever acceptance probability $\mathcal{M}(\omega)$ or utility distribution $\hat{U}_\omega$ is changed for some outcome, only the associated value needs to be corrected and the optimal policy can then be updated in $O(\log n)$ operations. This process though, does not update the S_k and P_k values. This update can be done after applying this process in linear time. Note that we do not need to update the S_k and P_k values while calculating eeu_q values.

To find the best query to ask, we simply have to loop over all queries and for each answer apply the aforementioned update process once to find the associated optimal policy ($O(\log n)$). This means that the total complexity of the process of finding the best query with its associated EEU value is $O(m \log n)$.

5 Evaluation

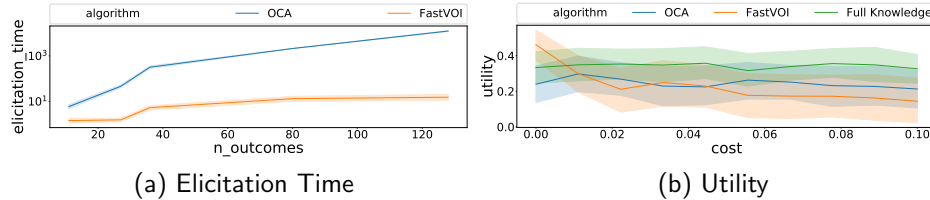


Fig. 1. The performance of different elicitation algorithms.

The main goal of this evaluation is to demonstrate the speed advantage of the proposed FastVOI algorithm. This section compares three agents: The OQA agent [3] representing state-of-the-art, the proposed FastVOI agent (section 4), and a *Full Knowledge* agent that shared the same structure as the three other agents but had access to the real utility function of its actor.

The set of scenarios that we used for comparison are taken from the ANAC competition [9] which started in 2010 and is still running annually. We used five bilateral negotiation scenarios from 2012 and 2013 competitions, namely: *Fifty-fifty* (a zero-sum scenario with 11 outcomes), *Laptop* (high number of win-win outcomes with 27 outcomes), *Flight-booking* (An integrative scenario having both good and bad possibilities with 36 outcomes), *Barter* with 80 outcomes, and *Outfit* with 128 outcomes.

Each scenario was run 10 times with the negotiation agent assigned to one of the two sides randomly for 10 different elicitation costs from 0 to 0.1. The initial utility distributions for all outcomes were uniform over the total utility range (0 to 1). The other agent accepted any of the outcomes in the top 25% of its own utility value with a probability equal to that utility after normalization. The elicitation agent did not know this information but had an acceptance model that assigned a probability of acceptance equaling the utility of the opponent. This simplified opponent and opponent model were chosen following [3] to focus the analysis on the elicitation process.

Fig. 1 shows the results of this experiment. At zero elicitation cost, all algorithms behaved similarly, nevertheless, with increased cost, all eliciting algorithms started to get less utility (Fig. 1-b). The differences between OQA and FastVOI were solely due to rounding errors and tie breaking with equal expected utilities and led to no statistically significant difference (t-test $t = 0.163$, $p = 0.871$). Considering execution time (Fig. 1-a), FastVOI was 3.61 times faster than OCA at 11 outcomes and 5947.6 times faster at 120 outcomes. These results support the theoretical analysis of section 4. Due to lack of space more detailed experiments with randomly generated scenarios will not be reported.

One limitation of the work presented in this paper — and almost all elicitation during negotiation in the literature [2, 10, 3] — is the inability to use the

internal structure of the utility function to guide the elicitation process. Utilizing the structure of the utility function will be one of our future research directions.

Another issue with all algorithms based on expected utility calculations is that they run the risk of offering some outcome early in the negotiation that leads to a very low utility specially in highly competitive scenarios (e.g. the Fifty-Fifty scenario). A reliance on minimizing maximum regret or a variant of the heuristic introduced in [10] for a simpler system may resolve this issue. Analyzing the effect of using an adaptive acceptance model is another direction of future research.

Conclusions

This paper presents an efficient algorithm for utility elicitation during negotiation based on rigorous analysis of the value of information provided with all possible queries. The proposed method achieves the same utility as the state-of-the-art algorithm while reducing the computational cost by a factor of $\frac{n}{\log n}$ which allows it to run orders of magnitude faster even for moderately sized problems. The proposed method was evaluated on realistic negotiation scenarios.

References

1. Aydoğan, R., Festen, D., Hindriks, K.V., Jonker, C.M.: Alternating offers protocols for multilateral negotiation. In: *Modern Approaches to Agent-based Complex Automated Negotiation*, pp. 153–167. Springer (2017)
2. Baarslag, T., Gerding, E.H.: Optimal incremental preference elicitation during negotiation. In: *IJCAI*. pp. 3–9 (2015)
3. Baarslag, T., Kaisers, M.: The value of information in automated negotiation: A decision model for eliciting user preferences. In: *Proceedings of the 16th Conference on Autonomous Agents and MultiAgent Systems*. pp. 391–400 (2017)
4. Chajewska, U., Koller, D., Parr, R.: Making rational decisions using adaptive utility elicitation. In: *AAAI/IAAI*. pp. 363–369 (2000)
5. De La Hoz, E., Marsa-Maestre, I., Gimenez-Guzman, J.M., Orden, D., Klein, M.: Multi-agent nonlinear negotiation for wi-fi channel assignment. In: *Proceedings of the 16th Conference on Autonomous Agents and MultiAgent Systems*. pp. 1035–1043 (2017)
6. Ha, V., Haddawy, P.: Problem-focused incremental elicitation of multi-attribute utility models. In: *Proceedings of the Thirteenth conference on Uncertainty in artificial intelligence*. pp. 215–222. Morgan Kaufmann Publishers Inc. (1997)
7. Ito, T., Hattori, H., Klein, M.: Multi-issue negotiation protocol for agents: Exploring nonlinear utility spaces. In: *IJCAI*. vol. 7, pp. 1347–1352 (2007)
8. Johnson, E., Gratch, J., DeVault, D.: Towards an autonomous agent that provides automated feedback on students’ negotiation skills. In: *Proceedings of the 16th Conference on Autonomous Agents and MultiAgent Systems*. pp. 410–418 (2017)
9. Jonker, C.M., Aydoğan, R., Baarslag, T., Fujita, K., Ito, T., Hindriks, K.V.: Automated negotiating agents competition (anac). In: *AAAI*. pp. 5070–5072 (2017)
10. Mohammad, Y., Nakadai, S.: Utility elicitation during negotiation with practical elicitation strategies. In: *IEEE SMC* (2018)